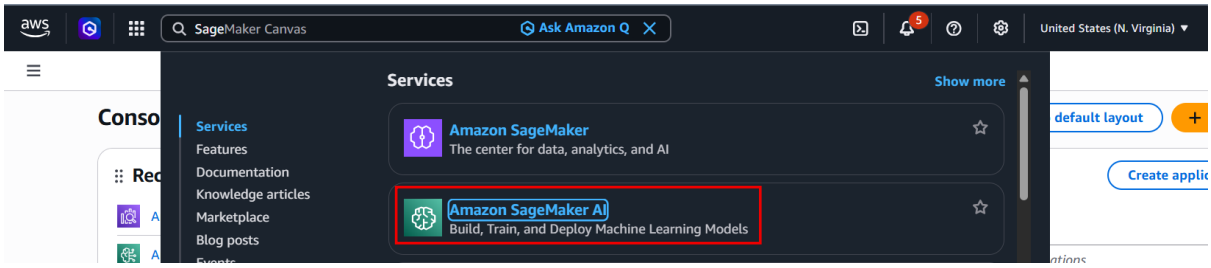
Setting Up SageMaker Notebook Instance

To Begin with the lab

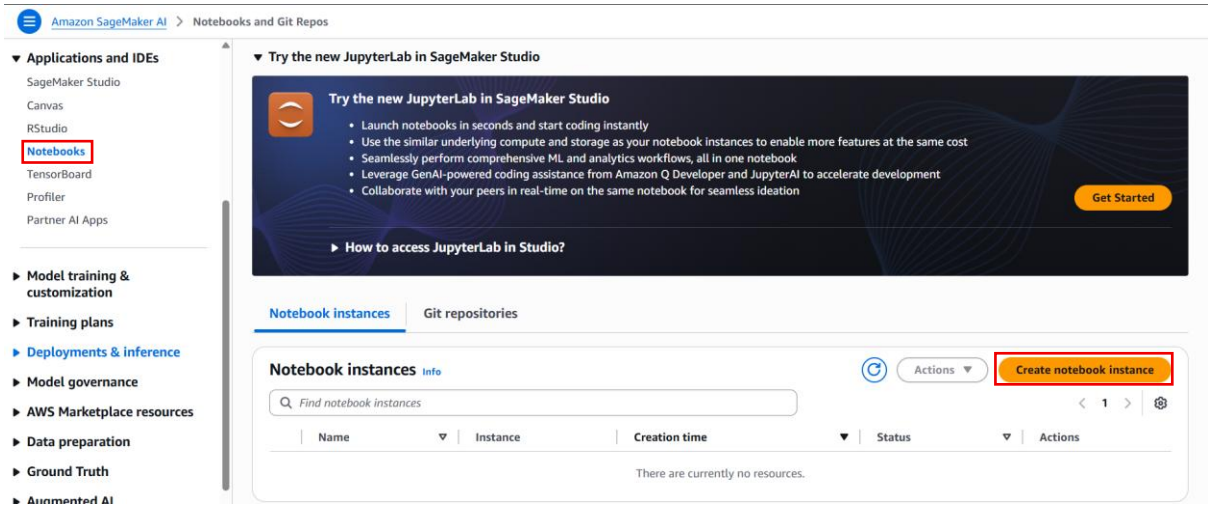
Summary of the Lab

Create an Amazon SageMaker notebook instance by selecting it in AWS, configuring a name, instance type (ml.t3.medium), and IAM role with S3 access. Launch and wait until "InService," then open JupyterLab. Create a PyTorch notebook, use cells for code and Markdown, document steps, and build, train, and evaluate a neural network model.

- Open Amazon SageMaker
- In the AWS search bar, type SageMaker
- Select Amazon SageMaker

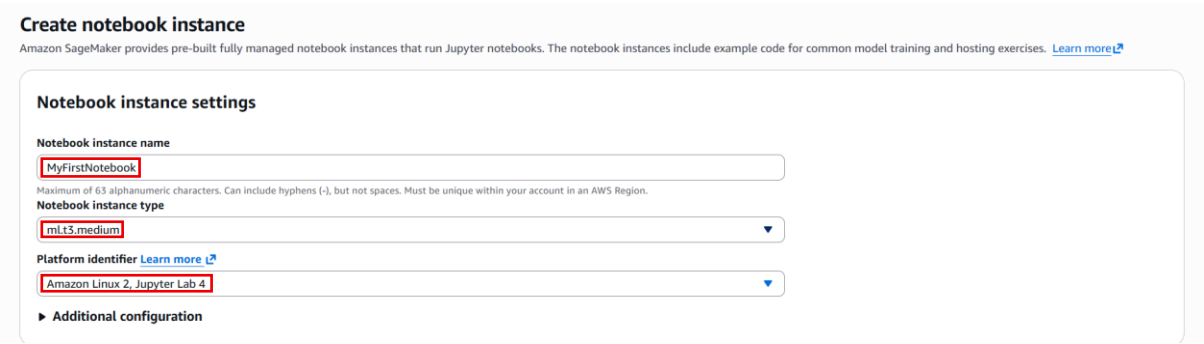


- Navigate to Notebook
- Click Create notebook instance

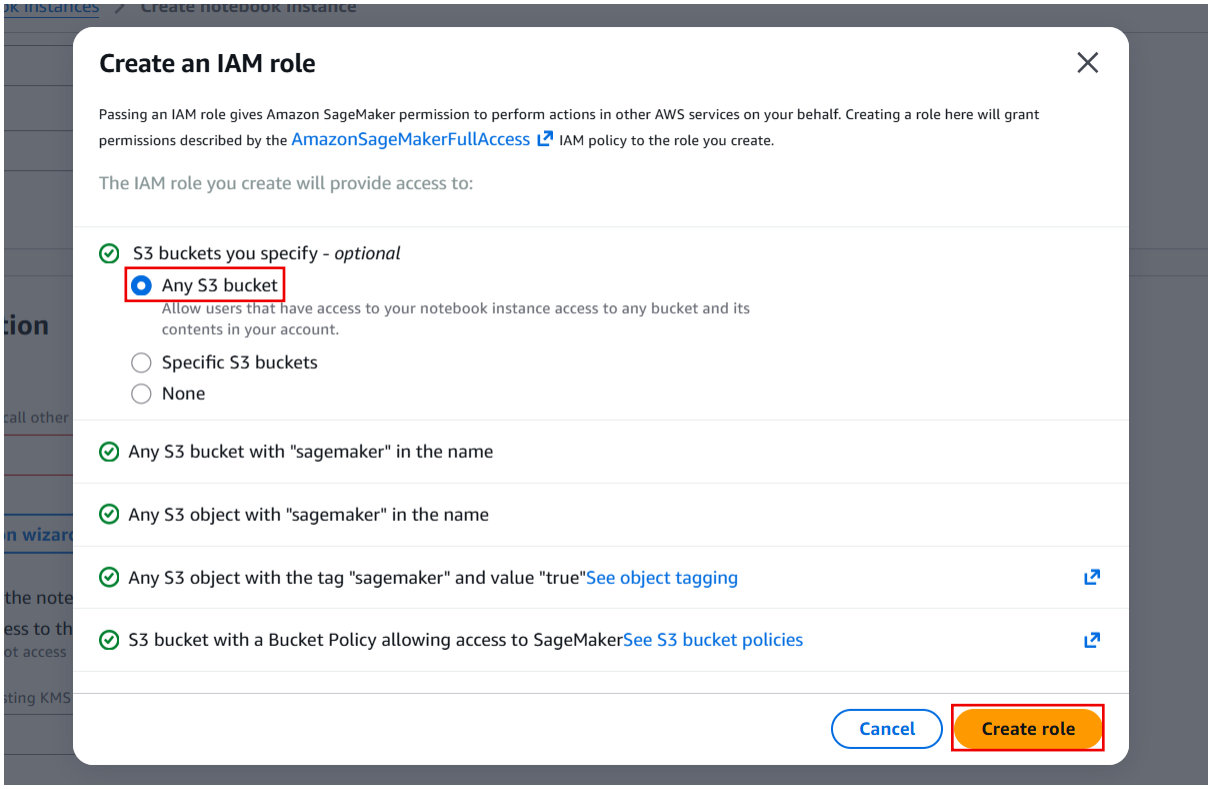


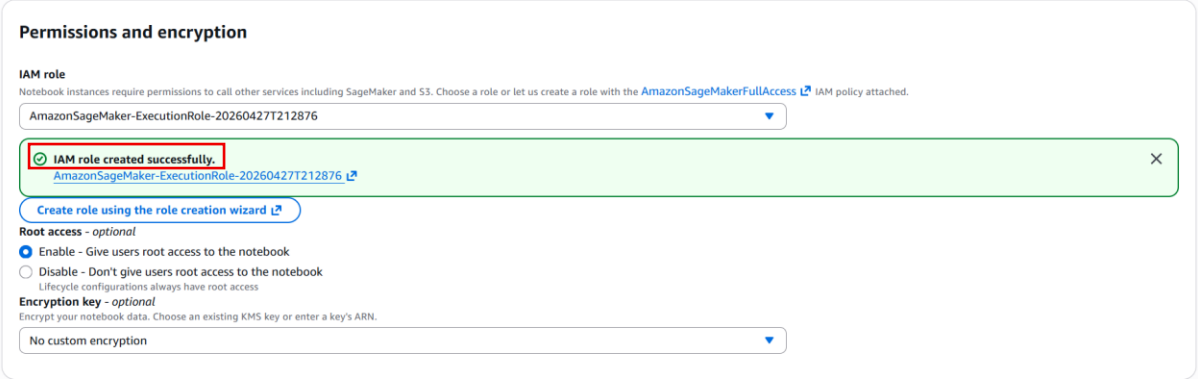
- Configure Notebook
- Fill in the required details:
- Notebook Name
    - Example: MyFirstNotebook
- Instance Type
    - Select ml.t3.medium
- Recommended because it is included in the AWS Free Tier (as stated in lecture)
- Platform Identifier

- Keep default:
    - JupyterLab 4

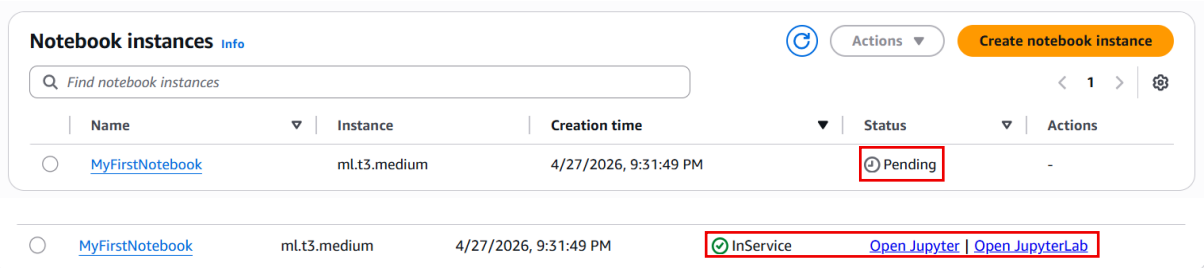


- Create IAM Role
    - Under Permissions, choose Create a new role
- Grant access to:
    - All Amazon S3 buckets (for lab/demo use)
- Click Create role

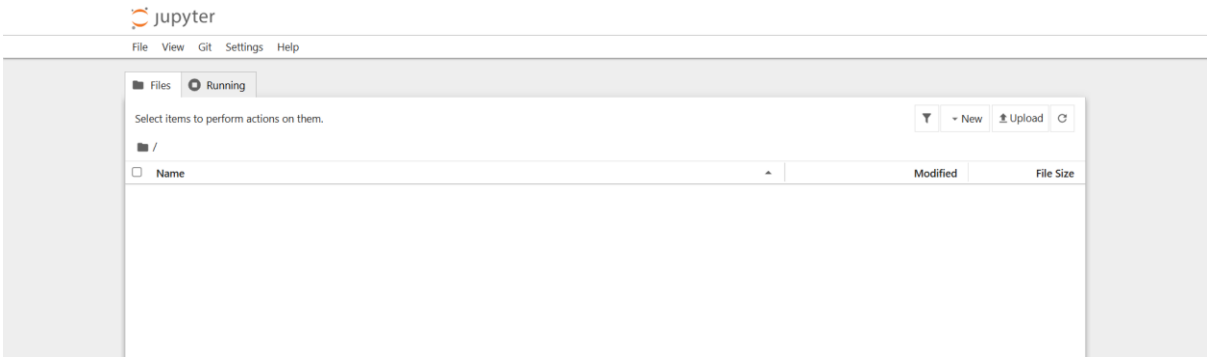




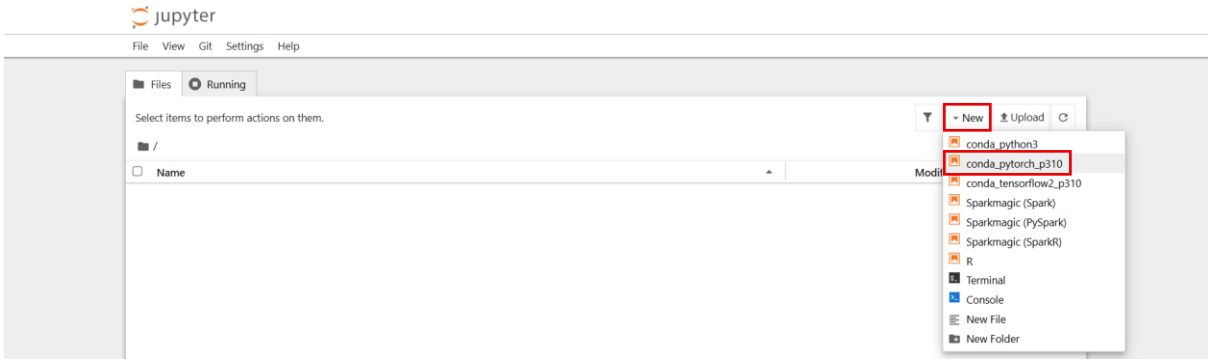
- Launch Notebook Instance
- Wait for Status Change
- Status will show Pending
- Wait around 3–5 minutes
- When status changes to InService, notebook is ready



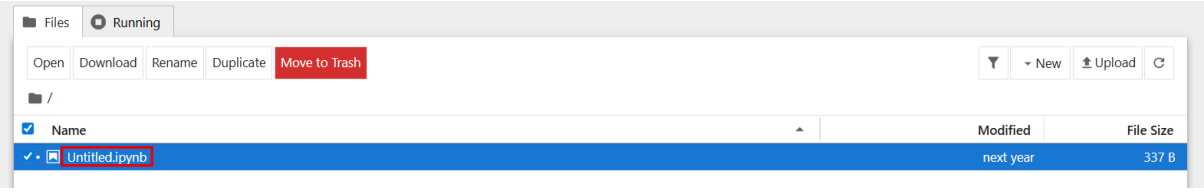
- Click Open Jupyter
- You can now:
- Create notebooks
- Write code



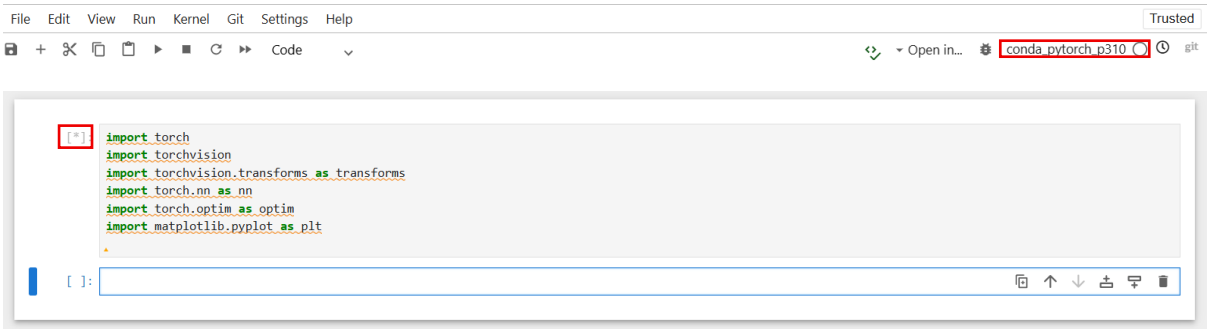
- Create a New Notebook
- Click New.
- Select a kernel.
- Choose PyTorch Kernel
- Select the PyTorch notebook kernel.
- This kernel includes preinstalled libraries such as:
- PyTorch
- TorchVision



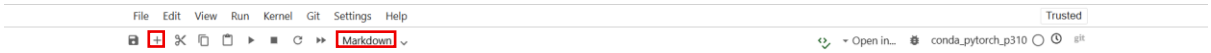
It has created a new file with the name 'untitled.ipynb'



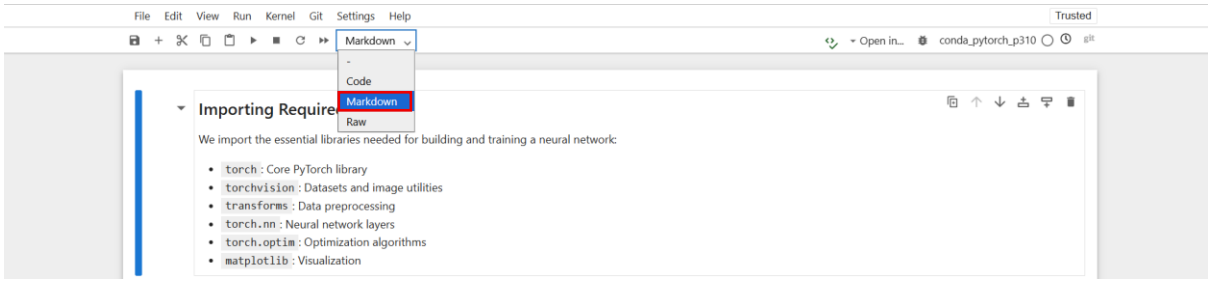
- Understand Notebook Interface
- Notebook consists of cells.
- Each cell can run code independently.
- Kernel status:
- Idle = not running code
- Busy/star icon = executing code



- Add Markdown Documentation
- Click + to add new cell.
- Change cell type from Code to Markdown.



This is a markdown cell



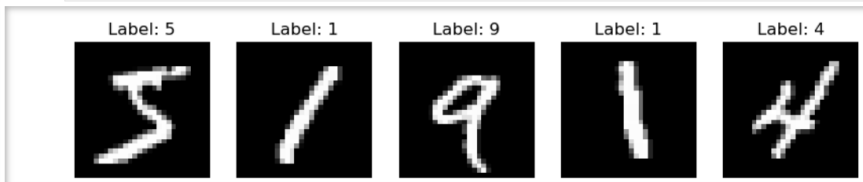
Load and preprocess the dataset, visualize sample images.

```
[3]: # Define transformations and load the dataset
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])

trainset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)

testset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=64, shuffle=False)
```

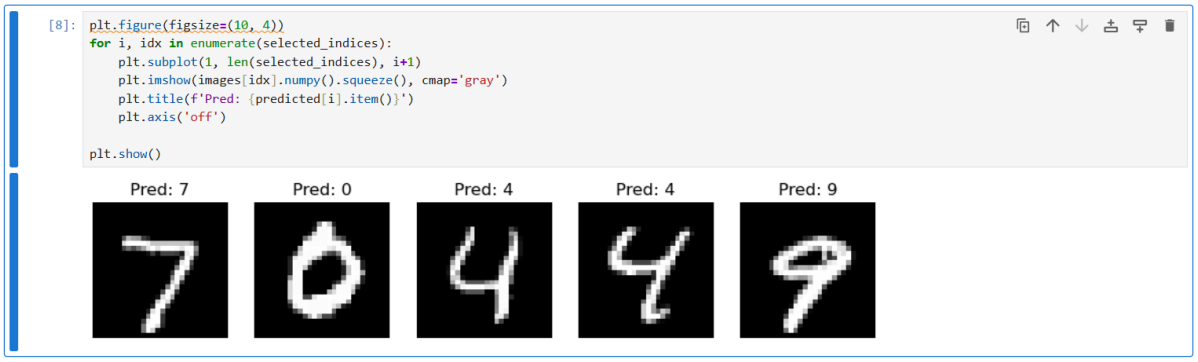
```
100%|██████████| 9.91M/9.91M [00:00<00:00, 207MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 13.4MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 72.5MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 2.19MB/s]
```

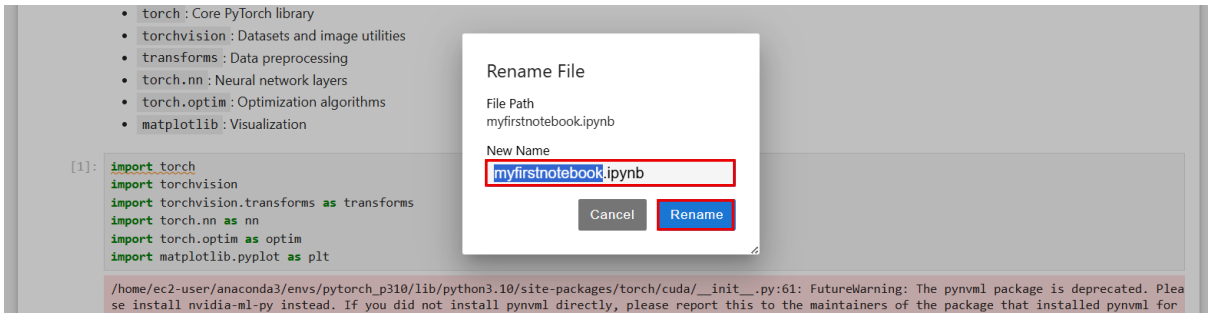
then build and train a neural network model using a defined loss function and optimizer.

```
[4]: # Create a DataLoader without shuffling
trainloader_no_shuffle = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=False)

# Get a batch of training images
dataiter = iter(trainloader_no_shuffle)
images, labels = next(dataiter)  # Get the first batch of images

# Select specific indices of images you want to use
selected_indices = [0, 3, 4, 6, 9]  # Example indices

# Display the selected images and their labels
plt.figure(figsize=(10, 4))
for i, idx in enumerate(selected_indices):
    plt.subplot(1, len(selected_indices), i+1)
    plt.imshow(images[idx].numpy().squeeze(), cmap='gray')
    plt.title(f'Label: {labels[idx].item()}')
    plt.axis('off')
plt.show()
```



Finally, use the trained model to make predictions and compare them with the actual labels.



- Rename Notebook
- Change notebook title to something meaningful.
- Click on the untitled to rename it.

- Example: FirstNotebook